GUPPI – Vergleich der lokalen Modelle

Stand 12.10.2026 · Daten aus GUPPIs eigenen Messungen (Standardtest *bench_std*, DevVM-Bauläufe mit versteckten Tests, Kontext-Leiter, Tempo-Messungen, Kleinteam-Test). Nichts geschätzt: Wo ein Wert fehlt, steht „–".

Hardware: NVIDIA GeForce RTX 4070 Ti (12 GB VRAM, Treiber 617.42) · AMD Ryzen 7 7700X (8 Kerne) · 31 GB RAM · **Engine:** GUPPIs eigener llama.cpp-Server b11160 (CUDA 13) · **Standard-Einstellung seit 11./12.10.:** Flash Attention an, KV-Cache q8_0, ubatch 512, 1 Slot · MoE-Modelle (Qwen3.6, Qwen3-Coder) legen einen Teil der Experten-Schichten in den Arbeitsspeicher (*n_cpu_moe*), daher sind sie langsamer als ihre aktive Größe vermuten lässt.

Kurzempfehlung
Hauptagent / Hauptcoder: Qwen3.6-35B-A3B (IQ4_XS) – bestes Gesamtergebnis, wenigste Werkzeugaufrufe, 131k Kontext. · **Schnelle Unteragenten:** Qwen3.5-9B Q6 mit MTP – ca. 109 Token/s, 131k Kontext, klein. · **Zweiter Coder für lange, sorgfältige Aufgaben:** Qwen3-Coder-30B-A3B. · **Prüfer / Leser langer Texte:** Gemma-4-12B. · **Kleinstaufgaben im Team:** Jan-code-4b + SWE-Review-8B (nur mit Test als Richter).

1. Übersicht: wofür jedes Modell gut ist

Modell (Datei)	Art · Größe	Rezept · Status	Gut für	Was es kann (Belege)	Schwächen
Qwen3.6-35B-A3B UD-IQ4_XS	MoE, 3B aktiv · 17,7 GB	coder.local_primary Standard-Coder	Hauptagent, Coder, Werkzeug-Arbeit, Selbstentwicklung	Standardtest in allen Solo-Läufen 4/4 (zuletzt 126–161 s, ca. 24 Aufrufe). Gebaut: Coder-Erweiterungen E-01…E-09, E-20, mcp_shield 4/4, core_boundary 4/4 (713 s), am009 3/3, e06 nach Nacharbeit 3/3.	Sehr große Stufen (am007, 30k Token Code) scheitern am 32k-Fenster; S0-Werkzeugprüfung vom 12.10. ohne Werkzeugaufruf (Prüfung wird noch untersucht).
Qwen3.5-9B Q6 mit MTP Qwen3.5-9B-MTP-GGUF/Q6_K	dicht, 9B · 7,2 GB	coder.llama_qwen35 aus, neu eingestellt	Schnelle Unteragenten, viele kleine Aufgaben, Zusammenfassen	Gleiche Gewichte wie Q6 unten plus MTP-Kopf: ca. 109 statt 56 Token/s, 80 % der Vorschläge angenommen. Kontext-Leiter sicher bis 131k.	Mit MTP noch kein Bau-Test. Rezept steht noch auf 16k-Grenze (Freigabe zum Anheben offen).
Qwen3.5-9B Q6 Qwen3.5-9B-GGUF/Q6_K	dicht, 9B · 7,0 GB	coder.llama_qwen35_q6 aus · GPU-Canary	Kleiner Coder, Canary-Modell (GPU-Selbsttest)	Standardtest 4/4 und 4/4 (102 s / 286 s), layer_summary 3/3 im ersten Anlauf, Dauertest 40/40, Nadeltest bis 131k.	Schwankende Laufzeit; früher (b11371, ohne FA) Treiberfehler ab ca. 24k – mit den neuen Einstellungen behoben.
Qwen3-Coder-30B-A3B UD-Q4_K_XL	MoE, 3B aktiv · 16,5 GB	coder.llama_qwen3_coder aktiv	Zweiter Coder, lange und sorgfältige Bauten	Standardtest 4/4 und 4/4 (ca. 265 s). Gebaut: am002–am005, am008 (Referenzbau 4740 s), e11 3/3 im ersten Anlauf. Kontext sicher bis 131k.	Langsamer als Qwen3.6; e10 nur 1/3; am006/am007/am009 nicht bestanden. Früher CUDA-Abstürze mit FA auf b11371.

Gemma-4-12B (QAT) Q4_0	dicht, 12B · 6,5 GB	verifier.llama_gemma Prüfer	Prüfer/Verifier, lange Texte lesen, Reviews	Standardtest 4/4 und 4/4; Codieraufgaben 92 % bei 32k und 64k; Kontext sicher bis 131k mit viel Reserve (2,5 GB frei).	Braucht viele Aufrufe (60–97 im Standardtest); eigener Bau e10 0/3; Denk-Ausreißer. Als handelnder Agent ungeeignet.
SWE-Review-8B i1-Q4_K_M	dicht, 8B · 5,0 GB	reviewer.llama_swe aus	Reviewer im Kleinteam	Im Team mit Jan: ca. 12 s je Review im Werkzeugmodus; 13 von 19 Urteilen richtig.	Kein verlässlicher Richter (falsche Ablehnungen und falsche Freigaben) – nur zusammen mit einem Test.
Jan-code-4b Q4_K_M	dicht, 4B · 2,7 GB	coder.llama_jan aus	Kleinstaufgaben, schnelle Einzelfixes	Mit Werkzeugen 10/10 einfache Aufgaben; im Team mit Test 16/16. Sehr schnell.	Standardtest nur 1/4; zu viele „Lektionen“ im Prompt verschlechtern es; nicht für größere Bauten.
Qwen3-30B-A3B-2507	MoE · 17,3 GB	coder.llama_qwen3_30b_2507 aus	–	Standardtest 2/4.	Schwächer als Qwen3.6 bei gleicher Größe.
DeepSeek-R1-0528-Qwen3-8B	dicht, 8B · 4,7 GB	coder.llama_deepseek aus	–	Codieraufgaben 68 % bei 32k, Kontext sicher bis 64k (Messung 03.10.).	38 % bei 64k; nicht weiter getestet.
Qwen3-14B	dicht, 14B · 8,4 GB	coder.llama_qwen3_14b aus	–	Nur Kontextmessung: sicher bis 16k (VRAM-Grenze).	Zu groß für mehr Kontext auf 12 GB.

2. Token-Erzeugung und Kontext auf dieser Hardware

Server-Messung = Zeiten des llama-Servers selbst (reines Erzeugen bzw. Einlesen). **Bau-Durchsatz** = erzeugte Token ÷ Laufzeit eines ganzen Baulaufs (enthält Einlesen, Werkzeuge, Prüfungen) – nur zum Vergleich der Modelle untereinander, nicht mit der Server-Messung.

Modell	Erzeugen Token/s (Server)		Einlesen Token/s (Server)	Mess-Einstellung	Bau-Durchsatz Token/s	Kontext sicher bis	VRAM belegt / frei beim Maximum
Qwen3.5-9B Q6 + MTP	109,4		ca. 3.370	32k, 6k Prompt, 512 Token Code, 12.10.	–	131k	10,5 / 1,5 GB
Qwen3.5-9B Q6 (ohne MTP)	55,7		ca. 3.850	wie oben	49–55	131k	10,3 / 1,7 GB
Qwen3.6-35B-A3B IQ4_XS	44–47		316–339	32k, n_cpu_moe 26–28, ubatch 1024	18–36	131k	9,6 / 2,4 GB
Qwen3-Coder-30B-A3B	33–41		117–155 (ub 512) 380–498 (ub 2048)	49k, n_cpu_moe 36–48	13–28	131k	10,2 / 1,7 GB
Gemma-4-12B	–		–	keine reine Server-Messung	37–44	131k	9,5 / 2,5 GB
Jan-code-4b	–		–	keine reine Server-Messung	27–89	16k (Rezept)	–
SWE-Review-8B	–		–	ca. 12 s je Review	–	16k (Rezept)	–
Qwen3-30B-A3B-2507	–		–	–	22	32k (Rezept)	–

3. Standardtest und Bauläufe

Standardtest *bench_std*: dieselbe Aufgabe in 4 Stufen für alle Modelle, versteckte Tests entscheiden, Solo-Modus, 32k-Fenster. Werte der Nacht 11./12.10. (je 2 Läufe).

Modell	Standardtest (Nacht)	Dauer s	Aufrufe	Eigene Bauten (versteckte Tests)
Qwen3.6-35B-A3B	4/4 · 4/4 (ohne Coder-Erw.: 4/4 · 4/4)	161 / 126	23 / 24	mcp_shield 4/4 · core_boundary 4/4 · e06 1/3, nach Nacharbeit 3/3 · E-01…E-09, E-20 bestanden
Qwen3-Coder-30B-A3B	4/4 · 4/4	270 / 262	34 / 38	e11_lesson_cards 3/3 · e10_self_repo_guard 1/3 · am002–am005, am008 bestanden
Qwen3.5-9B Q6	4/4 · 4/4	102 / 286	30 / 28	layer_summary 3/3
Gemma-4-12B	4/4 · 4/4	206 / 475	60 / 97	e10_self_repo_guard 0/3
Jan-code-4b	1/4 (09.10.)	250	82	Kleinteam-Aufgaben 16/16 mit Test
Qwen3-30B-A3B-2507	2/4 (09.10.)	663	76	–

4. Entfernte Modelle (12.10.2026, Quarantäne RM-20261012T055725)

Modell	Größe	Gemessen	Grund für das Entfernen
Qwen3.6-35B-A3B Q4_K_M	22,1 GB	Erzeugen 41–48 Token/s, Standardtest 3/3 bestanden	Doppelt zur IQ4_XS-Variante, größer, nicht besser
Qwen3.8-27B (+ MTP-Datei)	17,8 GB	Standardtest 4/4 · 4/4, s0_conn_check 3/3; aber ca. 312 s je Standardlauf, Bau-Durchsatz ca. 4 Token/s	Dicht und zu groß für 12 GB (nur 36 von 64 Schichten auf der GPU), am langsamsten, Kontext nur 49k
gpt-oss-20b	12,1 GB	ca. 65 Token/s; Codieren 98–99 % bis 32k; Nacht: 0/4 (HTTP 500 bei Werkzeugaufrufen)	Keine Werkzeugaufrufe im Agentenbetrieb; bricht über ca. 40k Eingabe ein
DeepSeek-Coder-V2-Lite	10,4 GB	73–81 Token/s bei 8k, aber 0,3 Token/s bei 32k	Standardtest 0/4
Hermes-4-14B	9,0 GB	Bau-Durchsatz ca. 42 Token/s	Standardtest 1/4

Zurückgeholt: Qwen3.5-9B Q6 **mit** MTP-Kopf (7,7 GB) – war im Löschvorschlag fälschlich als doppelt geführt.

5. Vorbehalte

- Die Verbindungsprüfung S0 (Werkzeugaufruf, lange Argumente, Fenster füllen) vom 12.10. ist bei **allen** Modellen nicht bestanden, meist mit gleichem Fehler (Fenster-Füllung HTTP 400, lange Argumente). Das deutet auf die Prüfung selbst; sie wird noch untersucht. Die Rangfolge für den Agentenbetrieb gilt daher unter Vorbehalt.
- Werte aus verschiedenen Tagen und Engine-Ständen (03.10. b11371 ohne FA, ab 11.10. b11160 mit FA und q8). Die Kontext-Leitern der Hauptmodelle sind alle vom 11./12.10. mit den aktuellen Einstellungen.
- Bau-Durchsatz hängt stark von der Aufgabe und der Zahl der Werkzeugaufrufe ab; für reines Tempo zählt die Server-Messung.

Quellen: docs/MODELL_VERGLEICH_ERGEBNISSE.md, measurements/night_2026-10-11/summary.json, day_2026-10-12/queue.log, context_ladder_*_2026-10-1[12].json, mtp_tune_qwen35_2026-10-12.json, moe_sweep_*.json, smallteam_*.json, docs/LOGBUCH_AGENT_MANAGER.md.